

Computational Methods for Scientists

Fall 26

VALERIA BARRA

August 15, 2026

Contents

0.1	Syllabus	1
0.1.1	Logistics	1
0.1.2	Overview	1
0.1.3	Organization and course design	1
0.1.4	Student Learning Outcomes	2
0.1.5	Assessment, grading policy and schedule	3
0.1.6	GitHub	5
0.1.7	Course materials, programming languages and environment	6
0.1.8	Target audience	6
0.1.9	Classroom Behavior	6
0.1.10	Resources for students	6
0.1.11	Accommodation for Disabilities	6
0.1.12	Preferred Student Names and Pronouns	6
0.1.13	Academic Honesty	7
0.1.14	Use of AI	7
0.1.15	Sexual Misconduct, Discrimination, Harassment and/or Related Retaliation	8
0.1.16	Religious Holidays	8
0.1.17	Land Acknowledgment	8
0.2	Lectures	9
0.2.1	Environment	9
0.2.2	Acknowledgements	10
1	1) First Class: Reproducibility and Git	11
1.1	1. Reproducibility in computational sciences	11
1.1.1	What is reproducible code?	11
1.1.2	How can we make our code reproducible?	11
1.1.3	2. Git	11
1.1.4	3. Package environments and managers	15
1.2	4. IDEs or editors	19
2	2) The Linux Filesystem and commands	20
2.1	1. The Linux Filesystem	20
2.2	2. Some basic Linux commands	21
2.2.1	Exercise 2.1	22
2.2.2	Exercise 2.2	22
2.2.3	Exercise 2.3	22
2.2.4	Exercise 2.4	23

0.1 Syllabus

0.1.1 Logistics

Course name: Comp-526

Course term: Fall 2026

Class time: **M-W-F: 11:00 am - 11:50AM am**

Mode of delivery: **in person**

Location: **LH 347**

Instructor: Valeria Barra

Pronouns: (she/her/hers)

Email: vbarra [at] sdsu [dot] edu

Office location: announced on the course Canvas page

Office Hours: announced on the course Canvas page or by appointment.

Tip

Office hours are an important time for asking questions, solving problems, discussing broader academic and career strategies, and providing feedback so I can make the class serve your needs and those of people with similar experiences and interests.

0.1.2 Overview

Scientific computing and mathematical modeling (both deterministic and stochastic) are fundamental tools for the solution of problems arising in the study of complex systems, whether originating from the physical, chemical, or biological sciences, or of an economic and social nature.

Scientific computing is a broad term that describes the use of computers for scientific, medical, and engineering applications. This definition is often application-focused or domain-driven.

On the other hand, *Numerical Analysis*, is used to solve problems in analysis (i.e., with real numbers) by numerical, rather than symbolic means. Some examples:

- Solving linear and nonlinear equations
- Computing values of definite integrals
- Solving differential equations

This can be seen as a problem-focused description.

Numerical (or Computational) Methods are algorithms (methods) for solving mathematical problems in the service of applications. Some examples:

- The method of conjugate gradients for linear systems
- Gaussian quadrature for computing integrals
- The Runge-Kutta method for initial value problems

This is an algorithm-focused description. In reality, this area encompasses all three: based on an application, we formulate some mathematical problems and find an algorithm for it to solve it using computers.

0.1.3 Organization and course design

We will start by giving an introduction to version control and reproducibility, which are key aspects of modern computational sciences. The foundations learned during the first lecture will serve as the basis for the workflow that you will utilize throughout the semester for assignments and project submissions (it might be useful to review carefully the first lecture multiple times). Then we will introduce the Linux filesystem and some basic shell commands.

We'll proceed with the evaluation of functions and introduce the concepts of conditioning and stability, which are applicable to every topic we encounter.

Then we'll explore rootfinding, our first infinite algorithm, in which we'll learn about convergence classes and the fundamental challenge of writing a function that is correct for all well-typed inputs.

We'll move to the concepts of stability and backward stability.

Next up will be an introduction to linear algebra, interpolation, and then differentiation.

We'll move on to integration and finally numerical solution of differential equations.

Although we'll continue with new content, during the semester we will have two separate short modules on compiled languages programming: C and Fortran.

Towards the end of the semester, for your final projects, you can then form small teams of like interest and work on an original study (numerical experiments and interpretation, comparisons, etc.) or on contribution to be shared with the community. Studies and contributions can take many forms.

0.1.4 Student Learning Outcomes

Upon completing this course, students will be able to

1. contribute to collaborative software with the use of version control systems, such as `git`
2. formulate problems in science and engineering in terms of computational methods
3. evaluate the accuracy and performance of algorithms
4. diagnose ill-conditioned problem formulations and unstable algorithms
5. develop effective numerical software, taking into account stability, accuracy, and cost
6. communicate about the above using figures, numerical experiments, writing, and presentation
7. search for and understand relevant literature and documentation
8. write programs in Julia, C, and Fortran

Expectations

1. Enter with a growth mindset, practice adaptive coping, and nurture your intrinsic motivation
2. Attend class (in-person) and participate in discussions
3. Make an honest attempt at activities, projects, etc.
4. Interact with the class notebooks and read reference material
5. Individual or group projects

	Fixed mindset	Growth mindset
Approaching a challenge	I'm not smart enough to do this.	This will give me the chance to learn something new.
	I'm not going to get anything out of this; so, I'm not going to put in my best effort. [Offers an excuse for any failures.]	I'm intimidated by this, so I'll break it down into smaller, more manageable projects and tackle them one-by-one to help me learn.
	I shouldn't have to try this hard. Only dumb kids have to try in this class.	This may not work, and that's okay. I can try my best and learn from my errors.
Confronted by failure	My first idea didn't work. This is "impossible."	That didn't work. But I really want to solve this puzzle. I'll try it a new way.
	Why should I try? I'm not actually going to learn something from it. I am hopeless at this subject.	Every try will teach me something new, even if it doesn't work.
	This wasn't my fault. X ruined the experiment!	If I keep trying, and incorporating what I learn, I'll make progress.

0.1.5 Assessment, grading policy and schedule

This class will have some assignments and projects (midterm and final). The final projects can be individual or group projects (depending on the number of students registered) and will be agreed upon with the instructor. There will be a midterm and a final oral presentation for each project. Moreover, a final report must be delivered. Instructions about what is expected for both midterm and final presentations as well as for the final report will be provided.

Grading breakdown:

- Participation (can include engagement in class, attendance, use of office hours, etc) (5% - includes a graded quiz (1%) with due date **TBD**, by midnight (AOE))
- Assignment 1 (10%): due date **TBD**, by midnight (AOE)
- Assignment 2 (10%): due date **TBD**, by midnight (AOE)
- Assignment 3 (10%): due date **TBD**, by midnight (AOE)
- Assignment 4 (10%): due date **TBD**, by midnight (AOE)

AOE: Anywhere on Earth time zone. This time zone is 12 hours behind Coordinated Universal Time (UTC) and typically used for online submissions to accommodate people from different time zones. Given that this course is taught in person in the Pacific standard time, I use this not really to facilitate submissions from different time zones, but to allow people not to worry about submitting by midnight sharp. This does *not* mean that I encourage people working late. I actually advocate for a healthy work/life balance for everyone (students and instructors alike).

- **Midterm Project** (20%): The midterm/final project choice and proposal will need to be discussed with your teacher, before its submission (*not* via email). Please make sure to use plenty of Office Hours to discuss your midterm/final project Proposal before submitting it. This project is broken down in two deadlines. Please note that Canvas does not allow setting multiple parts with different deadlines for a given assignment. For the **Midterm Project**, I will set the **earliest of the two** deadlines on **Canvas**.
 - Part 1, due **TBD**, by midnight (AOE): Community project analysis and proposal.
 - Part 2, due **TBD**, by midnight (AOE): Community project contribution proposal and creation of an Issue.
- **Final Project** (35%): This project is broken down in three deadlines. Please note that Canvas does not allow setting multiple parts with different deadlines for a given assignment. For the **Final Project**, I will set the **latest of the three** deadlines on **Canvas**.
 - Part 1, due **TBD** by midnight (AOE): Creation of a Pull/Merge Request
 - Part 2, **TBD**: In class oral presentations.
 - Part 3, due **TBD**, by midnight (AOE): submission of Final Project Report.

Assignments will be distributed no later than a week prior to the due date.

The schedule is subject to change (the instructor will announce any changes).

Late submission and absences policy: If you submit your assignments late, there is an increasing penalty (10% off for up to 24 hours late, 20% off for 24-48 hours late). No assignments will be graded if submitted later than 48 hours late.

Grading scale: The following grading scale will be used

Default Canvas Grading Scheme

Grade By Percentage

Letter Grade	Range
A	100% to 94%
A-	< 94% to 90%
B+	< 90% to 87%
B	< 87% to 84%
B-	< 84% to 80%
C+	< 80% to 77%
C	< 77% to 74%
C-	< 74% to 70%
D+	< 70% to 67%
D	< 67% to 64%
D-	< 64% to 61%
F	< 61% to 0%

Grade questions/disputes: All questions regarding grading must be received no later than 5 days from the date grades are posted in Canvas.

Any student who cannot attend class or submit assignments by their due date for serious issues (e.g., medical emergencies) or participation in university activities (e.g., official university travel for conferences or sports) that can be *documented*, should communicate those to your instructor as soon as possible before the deadline.

Details on Midterm and Final projects

Why community projects? Nowadays, the vital role of scientific software in research is widely recognized. Moreover, modern science is often conducted by large, collaborative multi-institutional aggregate teams scattered across the globe. To facilitate work across teams and to promote partnerships and collaborations, it is crucial for students to be exposed to open-source community projects and learn the best practices and standards by making contributions via software. If you are still unsure, check this reference “Why contribute to open source?” from the How to Contribute to Open Source guide.

Community contributions and analysis Over the course of the semester, you will follow the development activities of an *active* open source project of your choosing. This should be a project with an active developer community, hopefully from multiple institutions, that discuss their rationale in public, such as a mailing list and/or GitHub/GitLab Issues and Pull Requests or Slack/Discord channels. You will write and present about the performance and capability needs of key stakeholders, the way project resources are allocated, their metrics for success, and any notable achievements made over the course of the semester.

Your Final Project will be to make a contribution to be merged by the project of your choice. Adding new examples and/or improving documentation are extremely valuable contributions, but you may also add features or improve implementations. You can also analyze an existing bug or working example, comment on the implementation performance, and compare with other similar implementations from other packages/libraries. Please respect the time of project maintainers and reviewers by learning about the project and its expectations and process, communicating in advance, and leaving plenty of time for multiple rounds of review and revision.

0.1.6 GitHub

We’ll use Git with Classroom50 for managing activities and feedback. Check their discussions board for any potential issues or down times.

Tip

If you don’t have a GitHub account, follow these instructions from the SDSU Research & Cyber-infrastructure website and link it to your SDSUId.

- Use a personal email account rather than the SDSU one, so that you won’t have problems accessing your GitHub account in the future.
- Choose your username wisely! Most likely you will use this again in professional settings in your career.

The **last commit** to the GitHub repository made before the deadlines will be the one used to determine your grade. Students can modify their submitted files as many times as they want before their due date without incurring in any late penalty fee. If students modify any submitted file after the posted deadline, they are subject to the late penalty fees outlined in the **Late submission policy** above.

0.1.7 Course materials, programming languages and environment

I will provide all free course materials and suggested readings on the class website. If you prefer to read a print-out version, please talk to me. I will primarily use Julia and Jupyter notebooks for slides and activities in class. This environment is convenient to work with, general purpose, and has extensive library support. It is possible to write fast code in Julia, though performance implications can be mysterious. C, C++, and Fortran are popular languages for writing production numerical software, sometimes called from a higher level programming language like Python. MATLAB is also popular for numerical computing, though it is a proprietary environment and lacks general-purpose libraries.

Most HPC facilities use a Linux operating system and many open source software packages and libraries will have the best documentation and testing on Linux systems. You can use any environment for your local development environment, or use the SDSU's JupyterHub to experiment and develop without a local install. If you have never logged in before, check SDSU's Research & Cyberinfrastructure resources for students.

0.1.8 Target audience

Students in computational science, applied mathematics, or a quantitative science or engineering field.

Catalog Prerequisites:

- MATH 252: Calculus III
- MATH 254: Introduction to Linear Algebra

Good to know:

- Matrix theory
- Integration
- Partial differentiation

0.1.9 Classroom Behavior

Both students and faculty are responsible for maintaining an appropriate learning environment in all instructional settings, whether in person, remote, or online. Those who fail to adhere to such behavioral standards may be subject to discipline. Professional courtesy and sensitivity are especially important with respect to individuals and topics dealing with race, color, national origin, sex, pregnancy, age, disability (visible or invisible), creed, religion, sexual orientation, gender identity, gender expression, veteran status, political affiliation, or political philosophy.

0.1.10 Resources for students

Every student is encouraged to read the SDSU Student Academic Success Handbook (includes essential information for students). Please, watch this video from the Basic Needs Center and Economic Crisis Response Team (ECRT).

0.1.11 Accommodation for Disabilities

If you think you may qualify for accommodations because of a disability, please contact SDSU Student Ability Success Center and make your faculty member aware in a timely manner so that your needs can be addressed. Please allow 10-14 business days for this process.

0.1.12 Preferred Student Names and Pronouns

We recognize that students' legal information doesn't always align with how they identify. Class rosters are provided to the instructor with the student's legal name. If you feel that the name that appears on the class roster does not reflect your preferred name or pronoun, let your faculty member know.

0.1.13 Academic Honesty

SDSU has strict codes of conduct and policies regarding cheating and plagiarism. Become familiar with the policy and what constitutes plagiarism. Any cheating or plagiarism will result in failing this class and a disciplinary review by the University. These actions may lead to probation, suspension, or expulsion.

0.1.14 Use of AI

This course requires you to complete various assignments that assess your understanding and application of the course content. You are expected to do your own work and cite any sources you use and collaborators (humans or not) appropriately. You are personally responsible for understanding and verifying the code that you submit and include appropriate documentation.

The California State University system requires instructors to report all instances of academic misconduct to the Center for Student Rights and Responsibilities. Academic dishonesty will result in disciplinary review by the University and may lead to probation, suspension, or expulsion. Instructors may also, at their discretion, penalize student grades on any assignment discovered to have been produced in an academically dishonest manner such as cheating and plagiarism as described on the Cheating and Plagiarism page.

In May 2024, the University Senate extended its definition of plagiarism to include the un-cited use of generative AI applications, specifically: “representing work produced by generative Artificial Intelligence as one’s own.” Academic freedom ensures that instructors are empowered to determine whether students may use genAI in their classes and to what extent. To minimize confusion, we report here a statement regarding the use of AI in this class.

Instructor Approved Use of LLMs: Students should not use generative AI applications, known as large language models (LLMs), in this course except as approved by the instructor. Any use of generative AI outside of instructor-approved guidelines constitutes misuse. Misuse of generative AI is a violation of the course policy on academic honesty and will be reported to the Center for Student Rights and Responsibilities. LLMs, such as OpenAI’s chatGPT, Microsoft’s Co-Pilot, Anthropic’s Claude, Meta’s Llama, Google’s Gemini, or VSCode AI assistant etc. are valuable tools, which are still in their infancy, that will likely transform how we teach, learn, and code. However, such LLMs are highly sensitive to the (often biased) data that they are trained on and prone to hallucinations leading to inaccurate and unreliable results. Hence, it is necessary for the user to have a firm grasp and understanding of the material.

- Work created by AI tools may not be considered original work and instead, considered *automated plagiarism*. It is derived from previously created texts from other sources that the models were trained on, yet doesn’t cite sources.
- AI models have built-in biases (ie, they are trained on limited underlying sources; they reproduce, rather than challenge, errors in the sources)
- AI tools have limitations (ie, they lack critical thinking to evaluate and reflect on criteria; they lack abductive reasoning to make judgments with incomplete information at hand)

Given these important ethical caveats, it is crucial for students to learn how to use these tools and other online resources (e.g., stackoverflow.com) responsibly. For my class

- You **must acknowledge and cite** use of examples and aids that you include in your assignments, whether from LLMs or other sources.
- You must clearly identify the use of AI-based tools in your work. Any work that utilizes AI-based tools must be clearly marked as such, including the specific tool(s) used. For example, if you use ChatGPT-3, you must cite "ChatGPT-5. (YYYY, Month DD of query). “Text of your query and answers”. Or you can simply share a link to your chat.
- You must not use AI-based tools to write commentaries/reports/essays, but use **your own words** for those.
- You must be transparent in how you used the AI-based tool, including what work is your original contribution.

- You must ensure your use of AI-based tools does not violate any copyright or intellectual property law.
- You must not use AI-based tools to cheat on assessments.
- You must not use AI-based tools to plagiarize without citation.

In order to prevent misuse of these tools and to ensure students are adequately learning the material, the Instructor may ask students in class or during office hours about certain topics covered in this course after they have been introduced and the students' answers will contribute to the overall assessment and grades. Instructors and graders/TAs may also use AI detector tools. If you are found in violation of this policy, you may face penalties such as a reduction in grade, failure of the assignment or assessment, or even failure of the course. Finally, it's your responsibility to be aware of the academic integrity policy and take the necessary steps to ensure that your use of AI-based tools is in compliance with this policy.

0.1.15 Sexual Misconduct, Discrimination, Harassment and/or Related Retaliation

SDSU is committed to fostering an inclusive and welcoming learning, working, and living environment. SDSU will not tolerate acts of sexual misconduct (harassment, exploitation, and assault), intimate partner violence (dating or domestic violence), stalking, or protected-class discrimination or harassment by or against members of our community. Individuals who believe they have been subject to misconduct or retaliatory actions for reporting a concern should contact the SDSU Title IX Office.

Please know that faculty and responsible employees have a responsibility to inform the Title IX Office when made aware of incidents of sexual misconduct, dating and domestic violence, stalking, discrimination, harassment, and/or related retaliation, to ensure that individuals impacted receive information about their rights, support resources, and reporting options.

0.1.16 Religious Holidays

According to the University Policy File, students should notify instructors of planned absences for religious observances by the end of the second week of classes. See the campus policy regarding religious observances for full details.

0.1.17 Land Acknowledgment

For millennia, the Kumeyaay people have been a part of this land. This land has nourished, healed, protected and embraced them for many generations in a relationship of balance and harmony. As members of the San Diego State University community, we acknowledge this legacy. We promote this balance and harmony. We find inspiration from this land, the land of the Kumeyaay.

0.2 Lectures

The class material includes Jupyter-based notebooks for use during class.

You can view them here, or open them to interact. You can use any environment for your local development environment, or use the SDSU's JupyterHub on the Instructional Cluster to experiment and develop without a local install. If you have never logged-in before, check SDSU's Research & Cyberinfrastructure resources for students. Once you are on the Instructional Cluster page, simply click on the **Launch JupyterHub** button.

If you decide to use SDSU's Instructional Cluster's JupyterHub, you can download each individual notebook from the class repository, under the lectures section on your machine, and once opened Jupyter on SDSU's JupyterHub you can select **Upload Files** and upload the desired notebook in the web app to interact with.

0.2.1 Environment

This explains how I configure my environment so you can experiment with the slides locally, or adapt these tools for your own use.

Install Required Packages

First of all, download and install Julia, Long-term support (LTS) release is recommended.

Moreover, to interact with the notebooks, you want to install JupyterLab (new, richer ecosystem), via

```
pip install jupyterlab
```

Once installed, launch JupyterLab with:

```
jupyter lab
```

and select the Notebook app.

You can also only install the classic Jupyter Notebook standalone app via

```
pip install notebook
```

and to run the notebook type:

```
jupyter notebook
```

And for these (using Julia in Jupyter) you can run

```
julia -e 'import Pkg; Pkg.add("IJulia")'
```

from your terminal, or first start a Julia session with

```
julia
```

and then type:

```
]add IJulia
```

go back to your Julia REPL using backspace and type

```
julia> using IJulia
```

and then

```
julia> notebook()
```

to run the notebook.

Additional resources You can open these notebooks as normal notebooks (continuous scroll). If you wish to use them as slides, one useful package is

- RISE: General editing and presentation of slides
 - Alt-R: Snap in and out of presentation mode
 - Shift-I: toggle slides
 - Shift-G: toggle fragment (incrementally revealed cells)

Best practices

- When saving, use `Kernel ->Restart & Clear Output` to keep the stored data and diffs in the notebook small.
- For your images, several formats will do. Just keep in mind that PDF files are not web-friendly. PDF images can be converted to SVG using `pdf2svg`, and the result will still look sharp (unlike PNG) no matter the scale or zoom.

0.2.2 Acknowledgements

For most of this course's materials, I owe huge thanks to my postdoc mentor, Jed Brown, Associate Professor at CU Boulder. For some exercises, I am also very grateful to Jeremy Thompson, Assistant Research Professor at CU Boulder.

Chapter 1

1) First Class: Reproducibility and Git

Today:

1. Reproducibility in computational sciences
2. Git
3. Package environments and managers
 - 3.1. Julia
 - 3.2. Python
4. IDEs or editors

1.1 1. Reproducibility in computational sciences

1.1.1 What is reproducible code?

“But it works on my machine!”

Code is reproducible if someone is able to easily re-run it and get the same results.

Why might code not be reproducible:

- software or packages are not installed
- software uses different versions
- different configurations (environment variables, optimization flags)
- differences between platforms: operating systems (Windows, Linux, MacOS), architectures (x86, ARM, GPUs)

Some of these are unavoidable (e.g. going from working locally on a Mac to the Linux cluster), but we can do our best to minimize these

- we’re not aiming for “bit-for-bit” reproducibility

1.1.2 How can we make our code reproducible?

We need to supply both the code, and the “environment” used to run the code, and we need to keep track of it as we run experiments.

1.1.3 2. Git

Introduction to version control with git

Version control Software (VCS) is also referred as SCM (Source Code Management) or RCS (Revision Control System) tools. Version control, also known as source control, is the practice of tracking and managing changes to source files. Version control systems are software tools that help teams of contributors manage changes to source code over time. There are several options: `git`, `CVS`, `SVN`, etc.

In this lecture, we will focus on `git` in particular.

Tip

First of all, if you want to install `git` on your machine, follow this GitHub guide.

Tip

If you are unfamiliar with `git` and version control, the following guides can be helpful:

- Atlassian (bitbucket) git tutorials. A set of tips and tricks for getting started with `git`.
- GitHub's git tutorials. A set of resources from GitHub to learn `git`.

We provide a brief guide here.

Tip

If you don't have a GitHub account, follow these instructions from the SDSU Research & Cyber-infrastructure website and link it to your SDSUId.

- Use a personal email account rather than the SDSU one, so that you won't have accessing your GitHub account in the future.
- Choose your username wisely! Most likely you will use this again in professional settings in your career.

GitHub Account

Identity First make sure `git` knows your name and email address:

```
$ git config --global user.name "Your full name"
$ git config --global user.email "your_email_address@example.com"
```

Community projects

- There are several open-source, community projects hosted on major platforms like GitHub, GiLab or BitBucket. These are collaborative projects, often led or maintained by volunteers or not-for-profit organizations.
- Here are a few I contributed to:
 - libCEED (main contributor)
 - PETSc (minor contributor)
 - ClimaCore.jl (main contributor)
 - ClimaAtmos.jl (contributor)
 - ClimaCoupler.jl (contributor)
- You may want to check out a few of these online communities, and if you think you want to propose any suggested changes to their repositories, you can propose your own suggestions
- Before proposing any contributions, when checking out open-source, community projects, make sure to read about their coding conventions or style guides
- If you recognize the need for a feature that you and others may use in your work, please consider contributing this feature yourself. First by opening an Issue, if one associated to this feature is not already open.
- Readability and accessibility are very important aspects to make a codebase inclusive. For instance, `snake_case` or `CamelCase` are more accessible than `nocase` for people who have vision problems.

Forks and branches Create your own fork of the desired repository and check out your copy. For example:

```
$ git clone https://github.com/sdsu -comp605/spring25.git
$ cd spring25
$ git remote add your_username https://yourusername@github.com/sdsu -comp605/spring25.git
```

What is the difference between using the `https` or the `ssh` protocol? If you use `https` the primary downside for most people is that you must enter your Git password/token every time you push. While it gets added to a cache, it's not configured to cache permanently (though this can be changed). With SSH keys, it just uses the key file on disk every time so you don't have to enter your Git password/token at every push. Check this guide to generate SSH keys and add them to the `ssh-agent`.

Now you have two remote repositories: **origin**, which is your fork (hosted on your cloud account), and **upstream**, which is the main `sdsu-comp605/spring25` repository.

When part of an organization, people who contribute to an open-source project do not generally fork the repository, but **clone** it directly on their local machine. If you are not sure about the differences between forking and cloning, check this helpful article.

Create a branch for your feature; this will hold your contribution:

```
$ git checkout -b <branchname>
```

Basic steps: checking out a remote repository and get updates

- When you start working on a new feature branch, make sure you start from **main** by running: `git checkout main`.
- When you create a new branch and check it out, as in `git checkout -b <branchname>`, a common convention is to make **branchname** something along the lines of `<yourname>/<short-description>`.
- Every time you want to get updates from a remote repository, use `git fetch` or `git pull`.
- Difference between `git fetch` and `git pull`: `git fetch` updates your remote-tracking branches. This operation is safe to run at any time since it never changes any of your local branches. `git pull` brings a local branch up-to-date with its remote version, while also updating your other remote-tracking branches. In the simplest terms, `git pull` does a `git fetch` followed by a `git merge`.
- If you are working in a very large collaborative environment in which a lot of people are pushing their work every day using different branches, you may want to use `git fetch` and add the `-prune` option so that you don't download stale branches (e.g., `git fetch -prune`), before pulling.
- To download upstream changes and update the copy of the repo on your local machine use `git merge` or `git pull` (which, again, combines `git fetch` and `git merge` in one single command, so it's more convenient).
 - Be careful: this might trigger a **merge** warning if you have some unsaved or uncommitted changes locally. Always commit (or stash) your work and have a clean `git status` before attempting merges.

Commit and contribute your own changes

- Once you have made any change to any file in the repository (e.g., you might have written some code or documentation, or updated files such as License, Contributing, etc.) inspect these changes by running `git status`.
- Commit all files changed: `git commit -a` or first add selected files via
 - `git add <file1><file2>` (or more briefly, `git add -u` to stage all updated files at once) followed by `git commit`.
- New or modified files can be added to a commit in the same way.
- Push feature branch to the remote for review: `git push origin <branchname>`

- Use descriptive, useful commit messages (you can either do this via the text editor or directly at command line via `git commit -m`).

Useful tips When cleaning up your local branches, some of the following commands might be useful:

- Show local and remote-tracking branches: `git branch -a`.
- Show available remotes: `git remote -v`.
- Show all branches available on remote: `git ls-remote`. Use `git remote show origin` for a complete summary.
- Delete a local branch: `git branch -D <branchname>` (only after merge to `main` is complete).
- Delete remote branch: `git push origin :<branchname>` (mind the colon in front of the branch name).

Additionally, when debugging or inspecting the code for some potentially problematic changes introduced, some of the following commands can be used:

- Show logs: `git log`, or more advanced, `git reflog` (the latter includes any rebase and commit reshuffling you might have done).
- Show logs for file or folder: `git log <file>`.
- Show changes for each log: `git log -p` (add file or folder name if required).
- Show diff with current working tree: `git diff path/to/file`.
- Show diff with other commit: `git diff <SHA1>path/to/file`.
- Compare version of file in two commits: `git diff <SHA1><SHA1>path/to/file`.
- Show changes that are in `main`, but not yet in my current branch: `git log..main`.
- Discard changes to a file which are not yet committed: `git checkout <file>`. (If the file was already staged via `git add <file>`, then use `git restore <file>` first, and then `git checkout <file>` to discard local changes).
- Discard all changes to the current working tree: `git checkout -f`.

Push changes to your own repository If you want your changes on your local machine to go onto your online repository, while sitting on the `main` branch, do

```
git push origin main
```

However, it is **not** a good practice to work off the `main` branch.

Contributing to remote repositories To propose your contribution to any community project you want to create a so-called Pull Request (PR) in GitHub lingo or a Merge Request (MR) in GitLab lingo. The repository maintainers will review your proposed changes, make comments if needed, request further changes and reject or approve your contributions. See this GitHub documentation page on creating Pull Requests.

Code reviews Code reviews are possibly the most under-rated service in open-source/community projects. The best way to get familiar with a code base is reviewing other contributors' code.

Exercise 1.1 Initialize a git repository (you may call it `test`) via `git init`

Often, when we run a numerical experiment/example, there may be output files generated (plots, datasets, etc). We usually don't want to store the output in the repository, so we can tell git to ignore them. There are a couple of ways we can specify this:

- `.gitignore`: this is specific to this repository, and is checked into the repository
 - output or temporary files that are specific to this experiment
- `~/.config/git/ignore`: this is specific to your machine

- temporary files used by editors (`emacs`, `vim`) or more sophisticated Integrated Development Environments (IDEs) like `VSCode`
- `.DS_Store` files on Mac
- `.git/info/exclude`: specific to this repository *and* your machine (not checked into git)
 - occasionally useful

Files are specified according to patterns, e.g.

```
*.vscode # ignore all hidden files created by VS Code
!test.nc # with a preceding bang, this will exclude 'test.nc' from the gitignore
```

See Git manual: `gitignore` for more information

Exercise 1.2 Set up your `.gitignore` file in this `test/` directory

Don't lose your output If you run many numerical experiments, you don't want to overwrite your previous results (maybe that previous experiment was actually better than our most recent one!).

- A good practice for managing output of experiments and avoid output being overwritten is to concatenate parameters as strings in the output folder name so that each experiment will create a unique output folder, and this should facilitate experiments book-keeping.
 - But what if you want to repeat the same experiment with exactly the same parameters? This will overwrite your concatenated parameter string output folder name as well. Some people prefer using a time stamp (so that you are sure it is unique) but it might be obscure to visualize and remember which folder/experiment pair that output belongs to. Some other people use a combination of these two methods (time stamp + concatenated parameters string).
 - There is no *right* way of doing this, as long as you can find what works best for your own bookkeeping and be able to easily fetch an old result when needed (if you do research, sometimes you may be asked to show results from a couple of years ago!).
- Recommended reading: Taming the Chaos of Computational Experiments.

Suggested readings and resources

- Git official website
- Git cheatsheet
- Learn Git Branching (interactive/game)
- Git for computer scientists
- Git workflows
- Git book
- MOOC Reproducible Research and Data Analysis

1.1.4 3. Package environments and managers

3.1 Julia

Tip

If you haven't already, download and install Julia.

Julia is a relatively new programming language. Think of it as MATLAB done right, open source, and fast. It's nominally general-purpose, but mostly for numerical/scientific/statistical computing. There are great learning resources. We'll introduce concepts and language features as we go.

The Julia REPL Julia has an interactive command-line REPL (read-eval-print loop). See the dedicated REPL documentation page.

Different modes:

- The Julia interpreter:

```
julia> 1+1
2
```

- Help mode. Toggled by ?

```
julia> ? # upon typing ?, the prompt changes (in place) to: help?>
```

```
help?>
```

- The shell mode, toggled by ; and immediately followed by a shell command (if you are using Jupyter notebook; otherwise, you can go to a new line with the shell command)

```
julia> ; echo hello # upon typing ;, the prompt changes (in place) to: shell>
```

```
shell> echo hello
hello
```

- The package manager mode, toggled by]

Julia Pkg.jl The Julia environment is tracked with two files:

- **Project.toml**: contains “user-editable” information:
 - dependencies you use directly (e.g. loaded by `using` or `import`)
 - dependency version constraints (optional)
 - package metadata (optional: only if the project is also a package)
- **Manifest.toml**: contains the “derived” information necessary to recreate the environment exactly
 - all the recursive dependencies (i.e. the dependencies, their dependencies, etc.)
 - the exact version used (version number / branch name + git sha hash)

To be able to reproduce your code, you need to check in both files to git

To initiate a new environment, start `julia` from a directory of your choice with the `-project=.` option. If you don’t have a `Project.toml` file already in the current directory, this will automatically create an environment in your current working directory:

```
julia - -project=.
```

If instead, you want to use a different path from your current working directory in which a `Project.toml` already exists, use

```
julia - -project=<relative/path/to/Project.toml>
```

You can then recreate this environment on another machine by doing:

```
using Pkg
Pkg.instantiate()
```

or at the REPL:

```
]instantiate
```

- if you have the `Manifest.toml`, this will create an environment with the exact versions used

- if you don't have the `Manifest.toml`, this will resolve it (see below)

Specifying dependencies:

- `]add PackageName / Pkg.add("PackageName")`: this will add the most recent registered version
- `]add PackageName @version / Pkg.add(name="PackageName", version="version")`: use a specific version

Other useful Pkg operations:

- `]resolve / Pkg.resolve()`: creates or updates the Manifest if it isn't in sync with the `Project.toml` or its dependencies
- `]up / Pkg.update()`: update dependencies to their latest versions (subject to compatibility constraints)
 - This will modify the `Manifest.toml` file
- To create a new Julia project, create a directory for it and then activate that directory to make it the “active project”, which package operations manipulate:

```
]activate MyProject
```

- To use an existing environment local to a specific path rather than your default system environment, invoke `julia` with the `-project` flag, as in:

```
julia - -project=<path>
```

- To run a script from the command line using the environment in the current directory you can run

```
julia - -project=. myscript.jl
```

- Note that even if you do not specify the current directory with the period `.`, `julia` will try to find the environment to launch in the current directory, so `julia -project=.` is equivalent to `julia -project`

Once you have added new packages, to load a module from a package, you can use the `using` keyword.

Julia's *modules* are a unit of code in the Julia programming language that contain related functions, types, and data structures. They are used to organize code and to make it reusable in different parts of a project or in different projects. Modules help to reduce code duplication and improve maintainability.

The `using` statement is used to bring an entire module and its functions into your code, and it can be written as follows:

```
using MyModule
```

This statement will load the `MyModule` module and make all its functions available in your current code.

The `import` statement is used to bring specific functions from a module into your code. It can be written as follows:

```
import MyModule.myfunction
```

This statement will load only the `myfunction` function from the `MyModule` module and make it available in your current code.

You can also use the `import` statement to rename functions or modules, like this:

```
import MyModule.myfunction as func
```

This statement will load the `myfunction` function from the `MyModule` module and make it available in your current code as `func`.

In summary: With `using`, you can directly call `myfunction` without needing to qualify it with the `MyModule` prefix. On the other hand, `import` is used when you want to selectively bring in specific names from a module into your namespace. With `import`, you can choose which specific names you want to use, and you can rename them if you want to avoid naming conflicts.

3.2 Python

Analog reproducible environments in Python can be obtained via Conda. For example

```
conda env export > environment.yml
```

will give you an exact snapshot of the packages installed, which can be stored in version control. It can be reproduced with

```
conda env create -f environment.yml
```

Unlike Julia, Conda environments are *not* reproducible across platforms (e.g. MacOS =>Linux). And they can also export only the direct dependencies:

```
conda env --from-history export > environment.yml
```

but it may resolve differently. See Conda: manage environments: sharing an environment for more information.

pipenv (another Python environment manager) has `pipfile` and `pipfile.lock`: roughly analogous to `Project.toml` and `Manifest.toml`, but again the `pipfile.lock` is tied to a specific platform.

Containers



Containers are another way to create an isolated environment that can be easily shared. These include:

- Docker: widely used
- Singularity (now Apptainer): supported on several HPC systems.

1.2 4. IDEs or editors

We mentioned Integrated Development Environments (IDEs) briefly before. These are very fancy “editors”. Strictly speaking, you can modify code with any text editor. You don’t necessarily need to use an IDE. But using an IDE might make your life a lot easier and can help boost your productivity.

An IDE normally consists of at least a source-code editor, build automation tools, and a debugger. But they can also have nice features like syntax highlighting, code completion, refactoring, code search, and language support, just to name a few.

One of the most popular IDEs at the moment is VSCode.

VSCode supports many different “plug-ins” or extensions that can facilitate your work. One of the most useful one, if you want to collaborate live with others, is the LiveShare extension. LiveShare enables you to quickly collaborate in real time with a friend, classmate, or professor on the same code without the need to sync code or to configure the same development tools, settings, or environment.

For this course, I also recommend at least the `GitLens`, the `Julia`, and the `Jupyter` extension; but you may find other extensions useful.

Chapter 2

2) The Linux Filesystem and commands

Last time:

- Reproducibility and git
- Intro to Julia and its package management

Today:

1. The Linux Filesystem
2. Some basic Linux commands

2.1 1. The Linux Filesystem

The Linux file system can be intended in two ways: the directories *tree* that describes the file organization/structure and the format that defines how files are stored on disk (FAT32, ext4, etc). In this class, we will talk about the default layout of the directories tree on a Linux system.

Tip

Check this very useful resource on The Linux Filesystem

Directory	Description
/	Root directory of the entire file system
/bin	Essential command binaries needed for single-user mode
/boot	Boot loader files (e.g., kernels)
/dev	Device files (e.g., keyboard, mouse, etc; convenient to back up)
/etc	Specific system-wide configuration files
/home	Users' home directories, containing saved files, personal settings, etc.
/lib	Libraries essential for binaries in /bin and /sbin
/media	Mount points for removable media (e.g., CD-ROMs)
/mnt	Temporarily mounted file systems
/opt	Add-on application software packages (e.g., Word Procs, Accounting soft, etc.)
/root	Home directory for the root user
/run	Information about the running system since last boot
/sbin	Essential system binaries (e.g., fsck equivalent to Windows CHKDSK)
/srv	Site-specific data served by the system (e.g., Web servers data and scripts, etc.)
/sys	Information about devices, drivers, some kernel features
/tmp	Temporary files (usually not preserved between system reboots)
/usr	Secondary hierarchy for read-only user data (e.g., multi-user utilities and applications)
/var	Files whose context is expected to continually change during normal operation of the system (e.g., logs, spool files, etc.)

2.2 2. Some basic Linux commands

Tip

Check this very useful resource on Linux command line for beginners

Command	Meaning
<code>Ctrl l</code>	Clean screen (keeps command history)
<code>reset</code>	Initialize terminal
<code>Ctrl A</code>	Move cursor to the beginning of command line
<code>Ctrl E</code>	Move cursor to the end of command line
<code>ls</code>	List current working directory content
<code>ls -l</code>	List permission strings and other info
<code>ls -t</code>	Sort files and directories by their last modification time (most recently modified first)
<code>ls *.txt</code>	List all files with txt extension
<code>ls -al</code>	List <i>all</i> files (including hidden files)
<code>ls -l /</code>	List files in root directory
<code>pwd</code>	Print current working directory
<code>cd /</code>	Change working directory to root
<code>cd ..</code>	Change working directory one level up
<code>cd -</code>	Get back to previous directory
<code>cd ~</code>	Short hand for your home directory
<code>mkdir <dir_name></code>	Create new directory <code>dir</code> under current directory
<code>mkdir {dir1,...,dirN}</code>	(no spaces) Create new directories <code>dir1</code> , ... , <code>dirN</code> under current dir
<code>mkdir -p dir1/dir2/dir3</code>	Create a parent directory <code>dir1/dir2/dir3</code> (three levels deep)
<code>rm file1 ... fileN</code>	Remove files <code>file1</code> , ..., <code>fileN</code>
<code>rm -r dir</code>	Remove directory <code>dir</code> and its content recursively
<code>mv</code>	Move or rename existing files/directories
<code>cat file1 ... fileN</code>	Concatenate files <code>file1</code> ,..., <code>fileN</code> and print result
<code>touch file1 ... fileN</code>	Create empty files or edit existing file
<code>./<command></code>	Execute a local command/program/script
<code>chmod file</code>	Change file permissions
<code>man <command></code>	See documentation of any <code><command></code>

2.2.1 Exercise 2.1

Print the current working directory

Tip

If you want to use any shell commands in your Julia Jupyter notebook, you can simply change the mode of the Julia REPL by prepending `;` to your command.

2.2.2 Exercise 2.2

Create a new directory and move into it

2.2.3 Exercise 2.3

Now that you have created a directory, initialize a `git` repository (Exercise 1.1. from last class)

2.2.4 Exercise 2.4

Delete the newly created directory